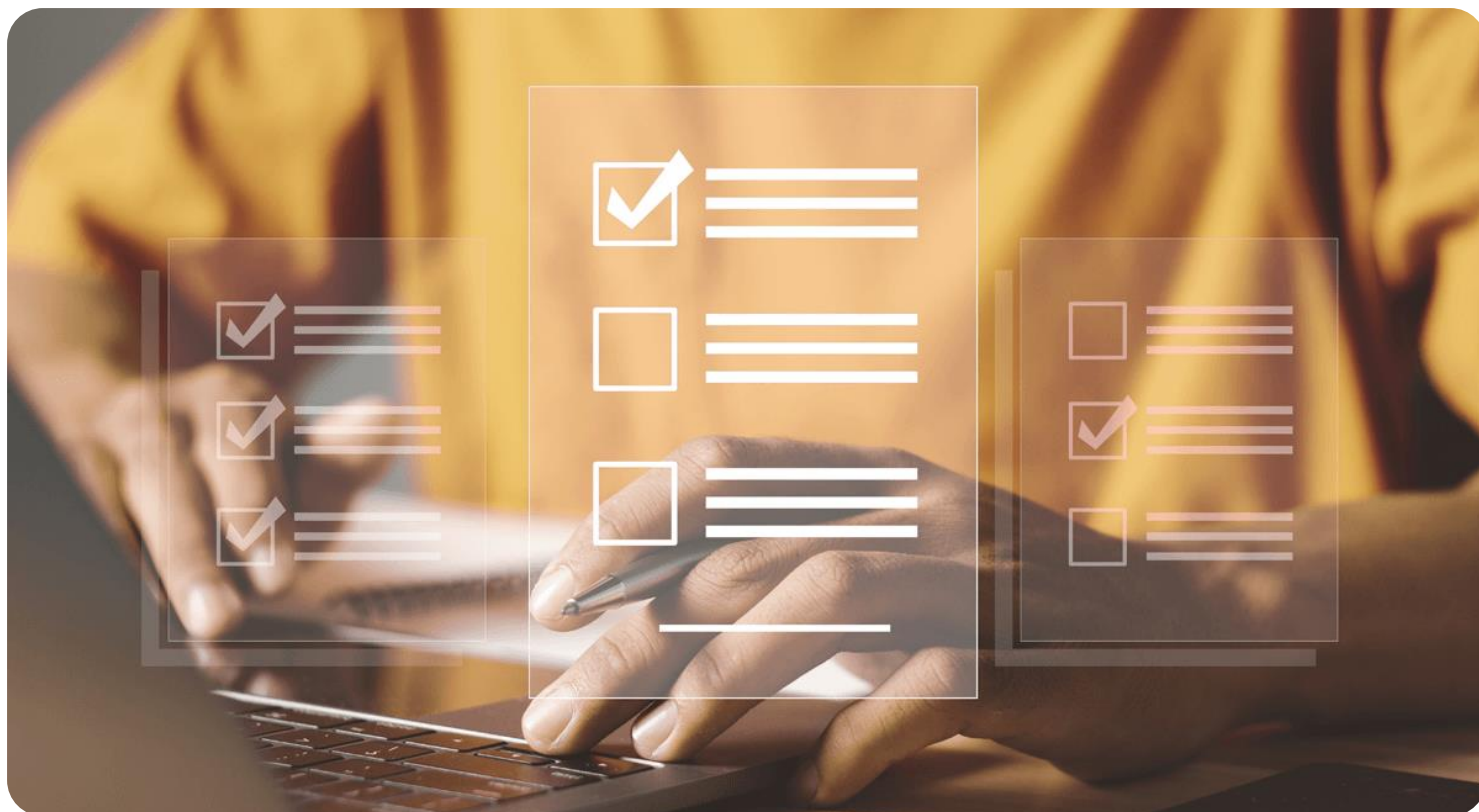




Semana 5

Desarrollo Backend III (PBY2203)

Nombre estudiante:	Jorge Germán Gallardo Heck
Carrera: Analista Programado Computacional	
Profesor: Alonso Esteban Castillo Pizarro	Fecha: 14/09/26

Informe con evidencia de ejecución

Estrategia elegida

Backend Central + 3 BFF independientes, uno por canal (Web, Mobile, Cajeros), cada uno como servicio Spring Boot separado, comunicándose vía HTTP/REST.

Backend Central: única fuente de verdad. Expone los datos de cuentas_bancarias, transacciones y movimientos_anuales tal como están en la base de datos, sin adaptarlos a ningún canal específico. Maneja autenticación (JWT), autorización por rol (CLIENTE/EMPLEADO) y autorización por propiedad de cuenta.

Sobre esta base neutral se construyó el endpoint de **dashboard**, que simula la vista de resumen de una página web: combina en una sola respuesta el detalle de la cuenta junto con sus transacciones y movimientos más recientes. A diferencia de los endpoints administrativos de listado completo (restringidos a EMPLEADO), el dashboard es accesible tanto para CLIENTE como para EMPLEADO, entendiendo que el CLIENTE es el usuario final, el dueño de la cuenta, que consume esta información como parte de su propia experiencia, no como una operación administrativa.

BFF: cada uno consume los mismos endpoints del Backend Central, pero decide cuánto detalle exponer según el contexto de su canal:

Canal	Nivel de detalle	Justificación
Cajero	Mínimo (solo saldo)	Espacio físico público, riesgo de exposición visual a terceros
Mobile	Intermedio (titular, saldo, tipo de cuenta)	Dispositivo personal, pero usado en espacios semi-públicos
Web	Completo (todos los campos de la cuenta, más un Dashboard con transacciones y movimientos recientes agregados en una sola respuesta)	Sesión en un entorno más controlado y privado. Al ser el canal de mayor capacidad de despliegue de información, agrega y consolida datos de múltiples fuentes para ofrecer una vista de resumen completa, evitando que el cliente web deba hacer múltiples llamadas por separado

Estrategia de identificadores: id interno vs. cuenta_id_legacy

El dataset de origen presenta `cuenta_id_legacy` duplicado entre distintas personas dentro de `cuentas_bancarias` (un mismo valor legacy identifica varias filas con nombres distintos), por lo que no es un identificador confiable para localizar una cuenta específica. Por este motivo, se utilizan dos identificadores con propósitos distintos:

- **id (parámetro de ruta, {id})**: el identificador autogenerado de la tabla (BIGSERIAL), único por fila. Es el que se usa en todos los endpoints (GET `/api/cuentas/{id}`, `/id/saldo`) para localizar una cuenta de forma inequívoca.
- **cuenta_id_legacy**: el identificador de negocio original del sistema legacy. Se usa exclusivamente para la validación de **propiedad de cuenta**: cada usuario tiene asociado un `cuenta_id_legacy` en la tabla `usuarios`, y al consultar una cuenta por id, el Backend Central compara el `cuenta_id_legacy` de esa fila contra el del usuario autenticado.

Esta segunda validación tiene una limitación conocida y aceptada: dado que varias filas pueden compartir el mismo `cuenta_id_legacy`, un usuario podría acceder a una cuenta distinta a la suya si ambas comparten ese valor. Se optó por esta solución de todas formas porque añade una capa real de control de acceso (bloquea el caso más común, donde los valores legacy difieren), documentando explícitamente su alcance en vez de omitir la validación por completo.

Manejo de excepciones

Cada servicio implementa un `GlobalExceptionHandler` (`@RestControllerAdvice`) que traduce errores técnicos a respuestas JSON consistentes, sin exponer detalles internos del sistema:

Código	Causa	Mensaje
404	Recurso no encontrado	"La cuenta solicitada no existe"
401	Token inválido, expirado o ausente	"Debes iniciar sesión para acceder a este recurso"
403	Rol insuficiente, o cuenta que no pertenece al usuario	"No tienes permisos para acceder a esta cuenta"
503	Backend Central no disponible	"El servicio no está disponible en este momento"

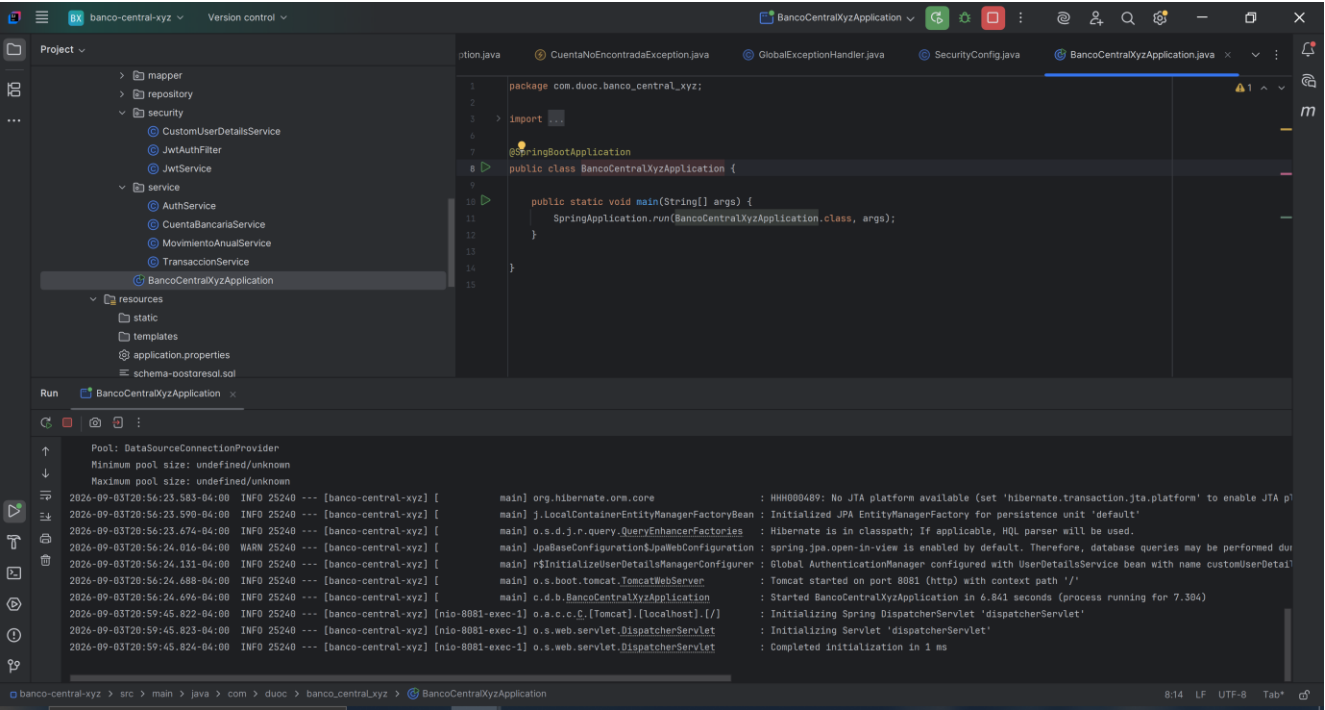
En los BFF, `CuentaBancariaClient` captura las excepciones HTTP (`HttpClientErrorException.NotFound/Unauthorized/Forbidden`, `ResourceAccessException`) y las traduce a excepciones propias del BFF — el Controller nunca queda expuesto a errores de red o HTTP crudos.

Puntos de conexión (puertos)

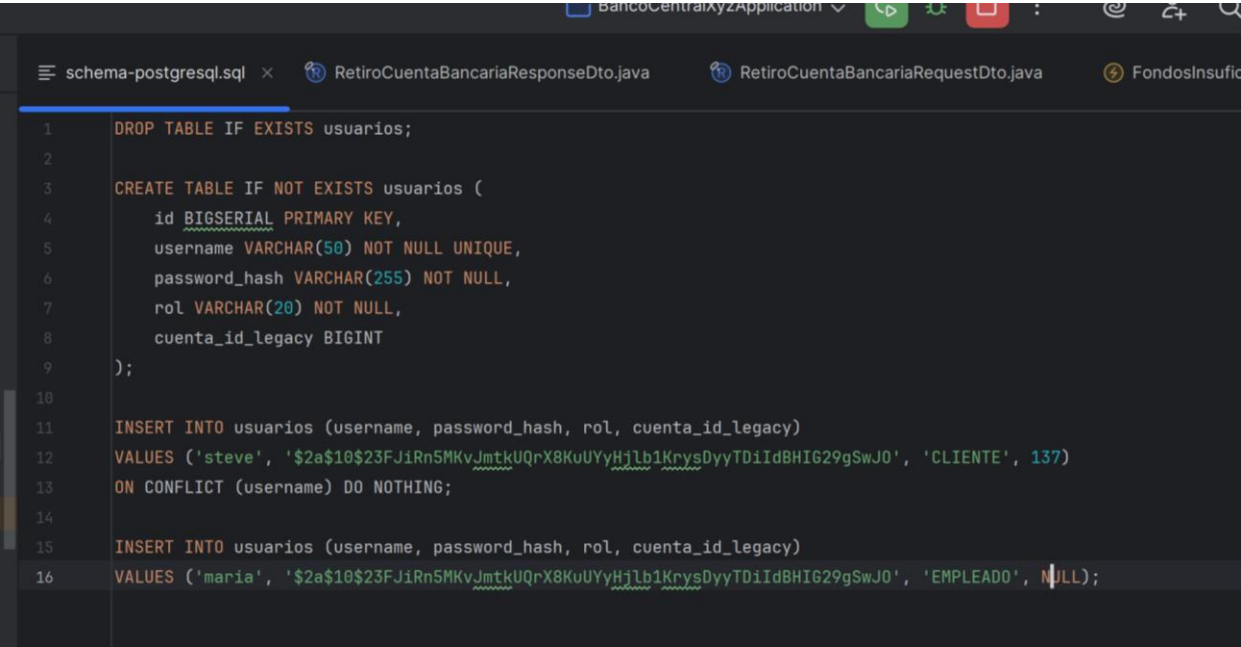
Servicio	Puerto	URL base
Backend Central	8081	/auth/login, /cuentas/{id}, /cuentas/{id}/mobile, /transacciones/recientes, /movimientos/recientes
BFF Cajeros	8082	http://localhost:8082/bff-cajero
BFF Mobile	8083	http://localhost:8083/bff-mobile
BFF Web	8084	/cuentas/{id}, /cuentas/{id}/dashboard

Evidencias

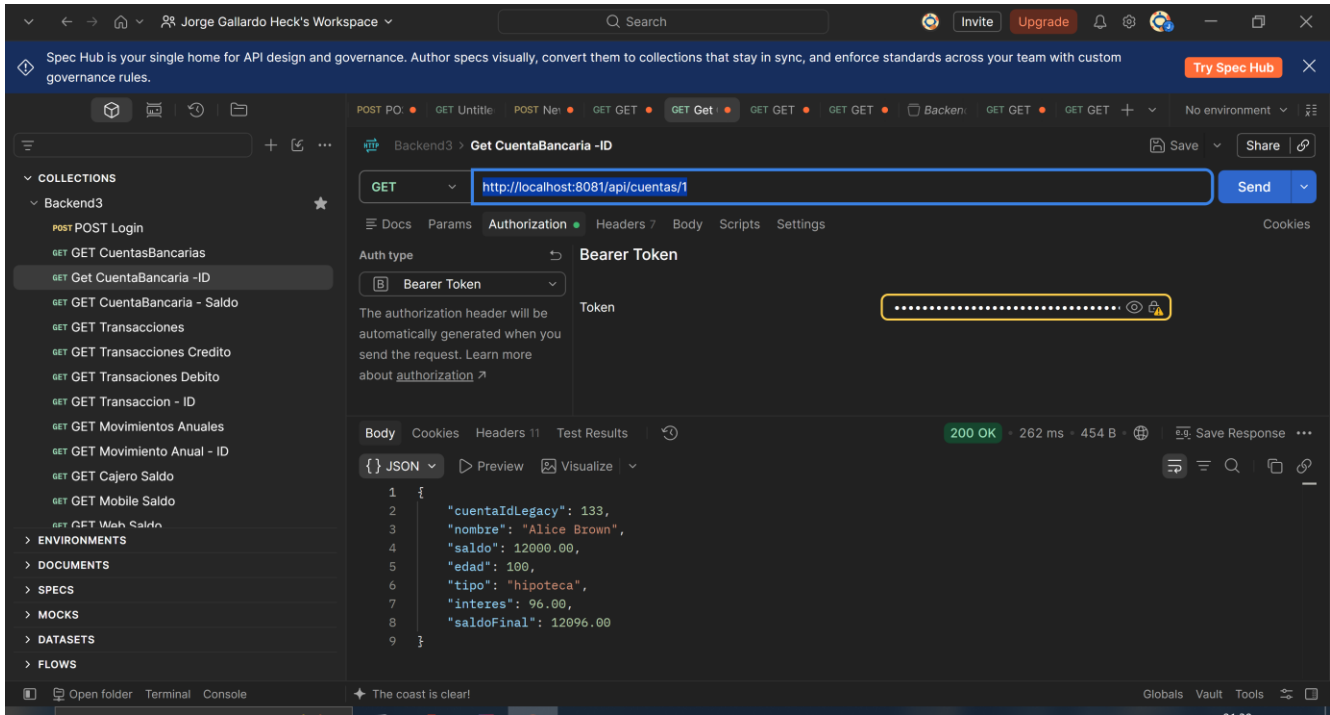
BancoCentral en ejecución:



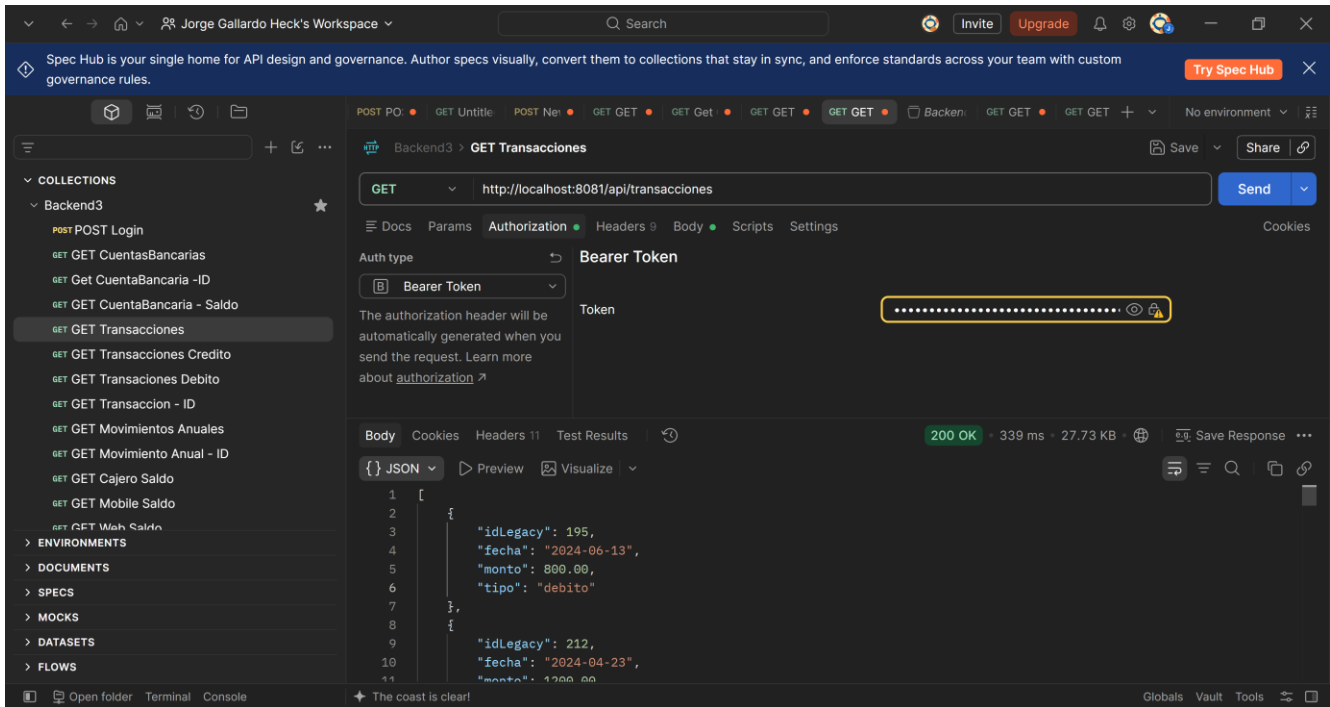
Usuarios a tabla usuarios



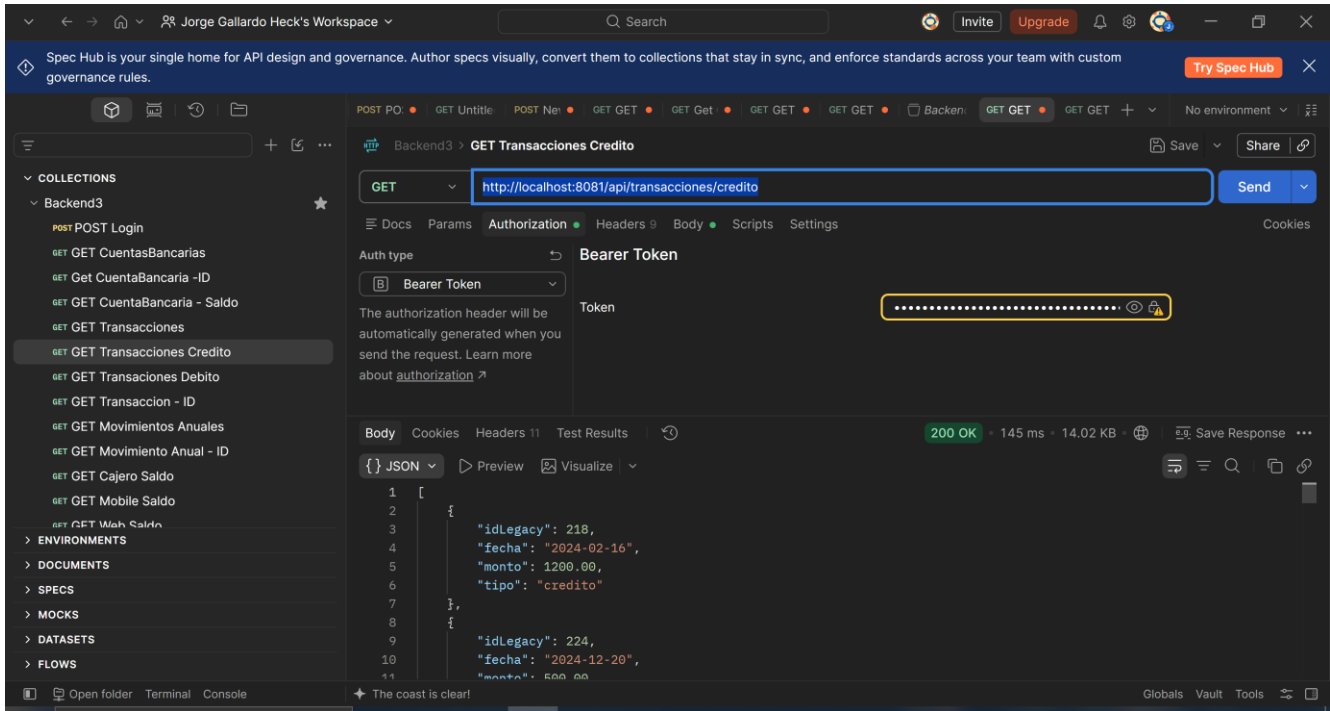
☐	id bigserial	username varchar(50)	password_hash varchar...	rol varchar(20)	cuenta_id_legacy bigint	+
☐	1	steve	\$2a\$10\$23FJiRn5MKvJmtk...	CLIENTE	137	
☐	2	maria	\$2a\$10\$23FJiRn5MKvJmtk...	EMPLEADO	NULL	



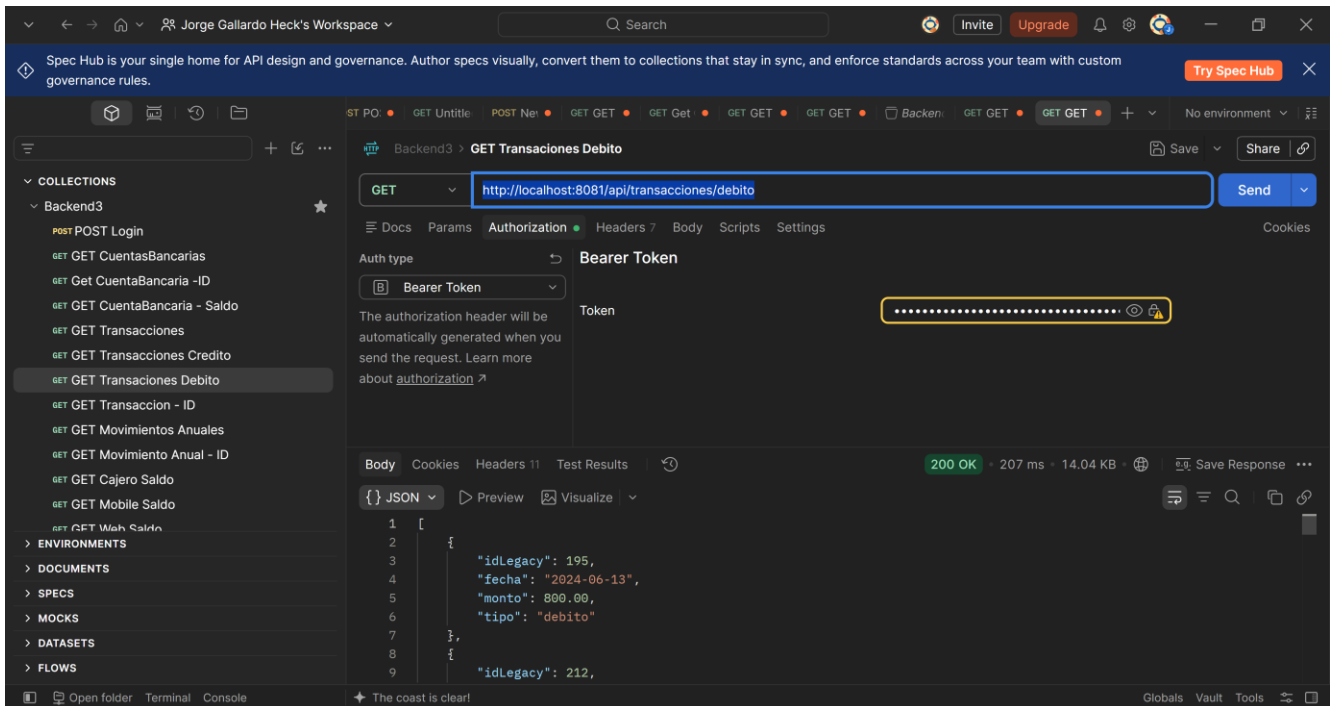
<http://localhost:8081/api/transacciones>



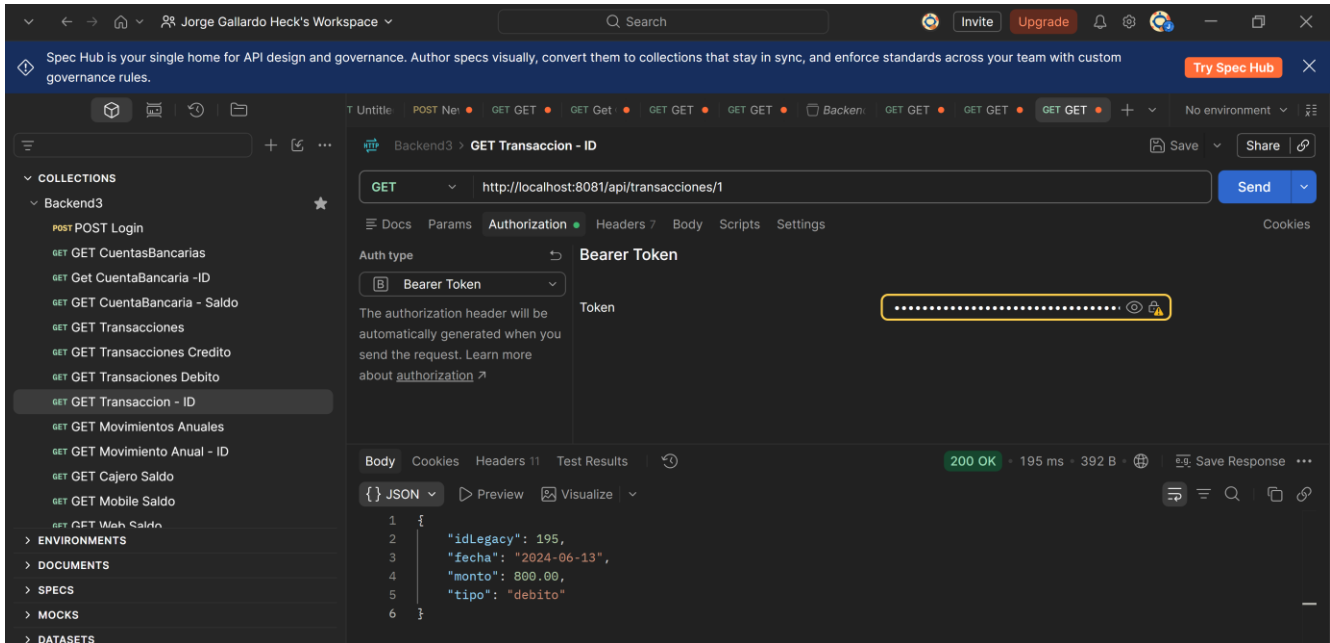
<http://localhost:8081/api/transacciones/credito>



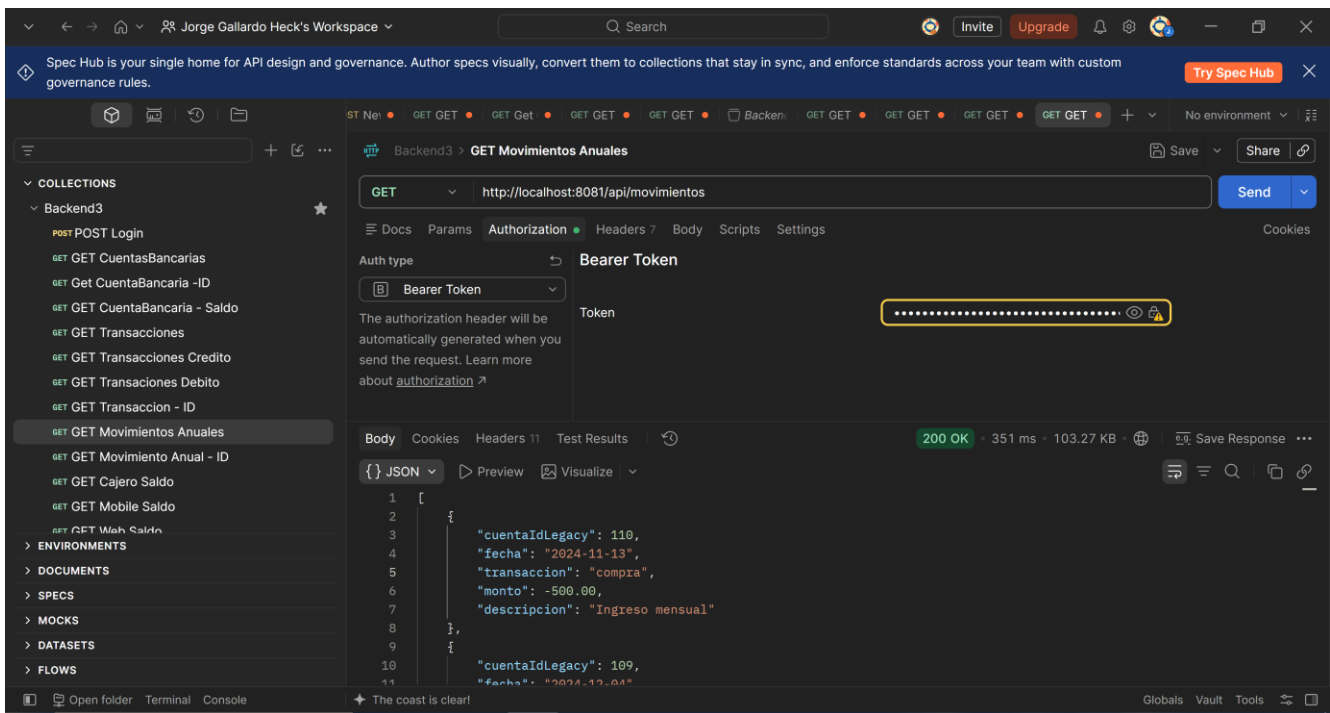
<http://localhost:8081/api/transacciones/debito>



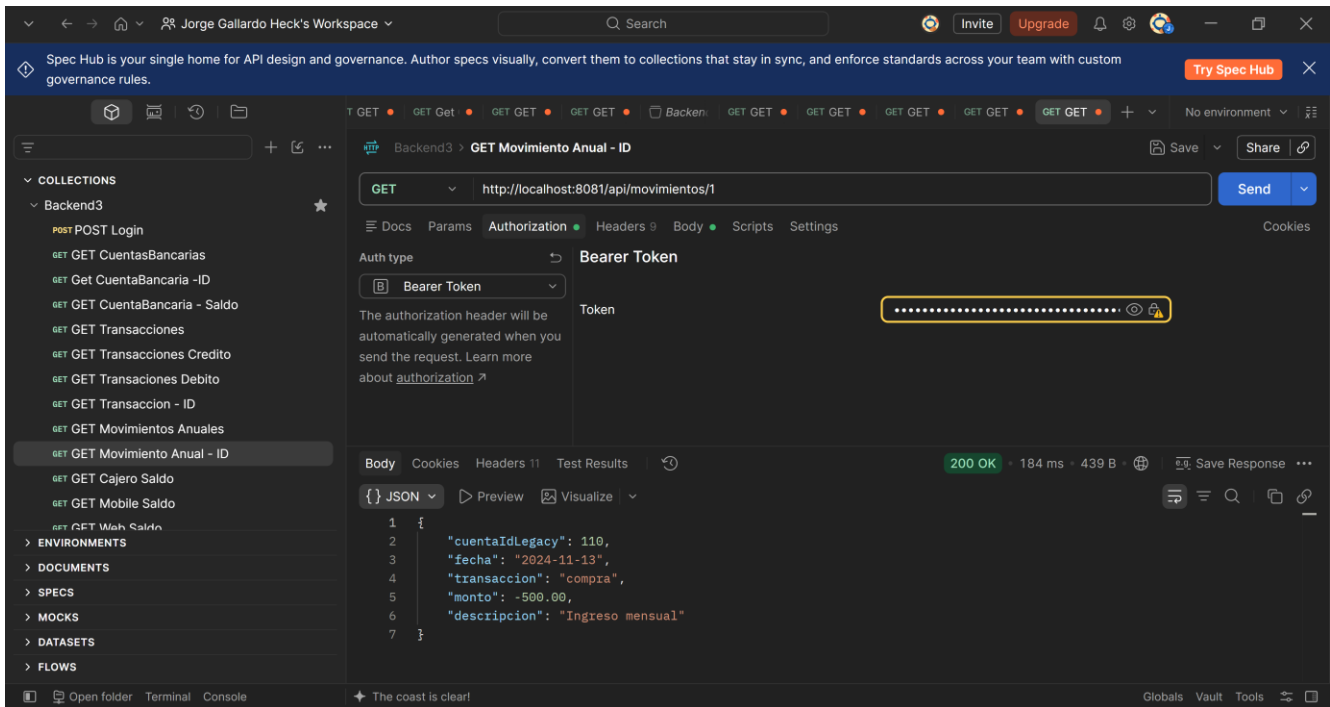
<http://localhost:8081/api/transacciones/1>



<http://localhost:8081/api/movimientos>



<http://localhost:8081/api/movimientos/1>



Se autenticó como el usuario steve (rol CLIENTE, cuenta_id_legacy=8 en la tabla usuarios), cuya cuenta real corresponde a la fila id=8 de cuentas_bancarias (Steve Rogers). Al solicitar GET /bff-cajero/cuentas/7/saldo (una cuenta con cuenta_id_legacy distinto a 8), el Backend Central detecta que la cuenta consultada no pertenece al usuario autenticado y lanza AccesoDenegadoException, que el BFF Cajeros traduce a una respuesta 403 con mensaje "No tienes permisos para acceder a esta cuenta"

🔖

🔗

console.neon.tech/app/projects/aged-thunder-83407194/branches/br-luc...

📄

🔍



🏠

banco_xyz Admin

🔍 Search...

OK

🟢 All OK

🟢 Ask AI

Tables

production

neondb

Schema

public

Search...

cuentas_bancarias

cuenta_id_legacy ASC

nombre ASC

DATA

STRUCTURE

<

>

Filters

Sort

Columns

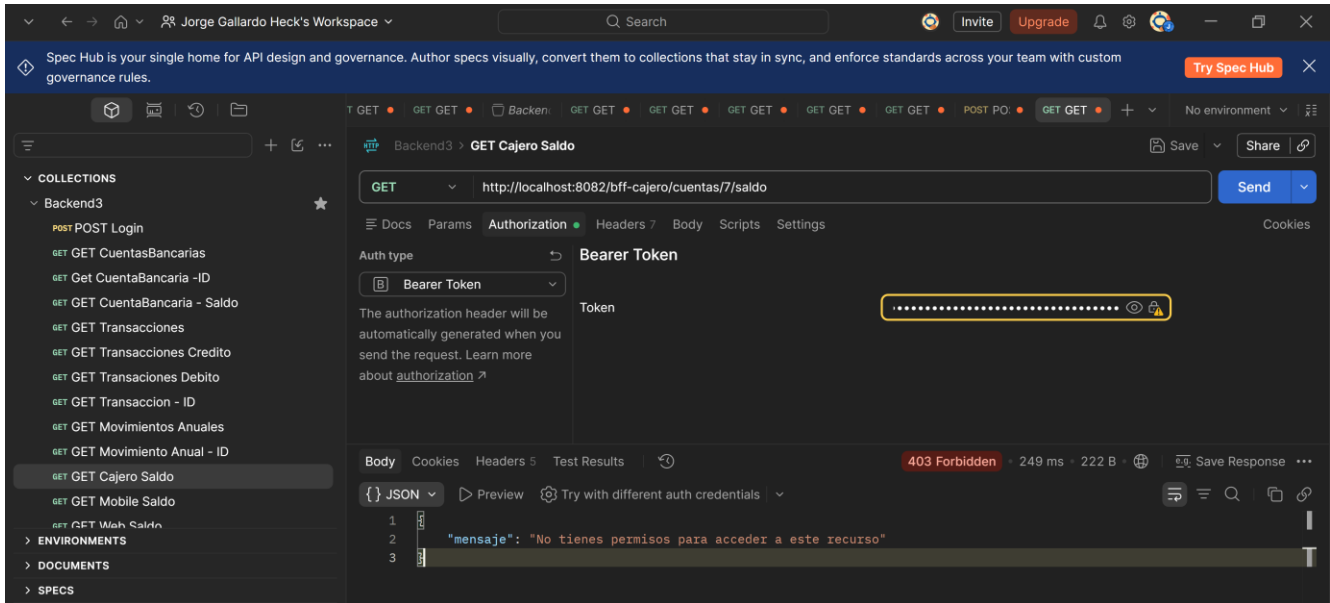
+ Add record

430ms

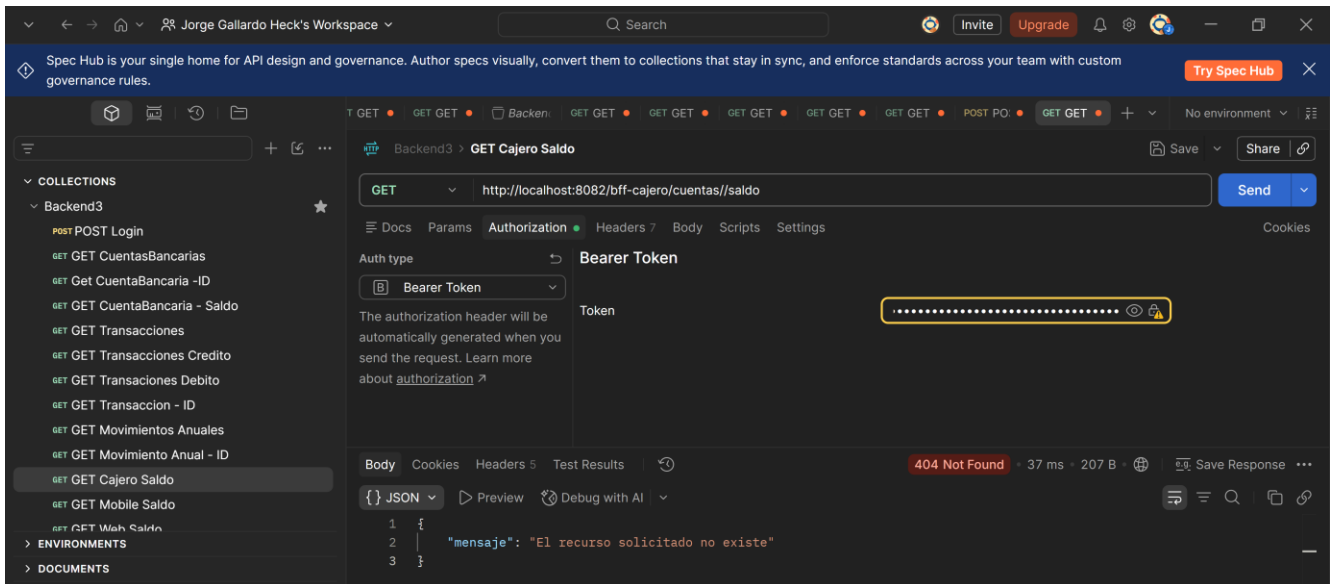
<

1 - 50 of 50

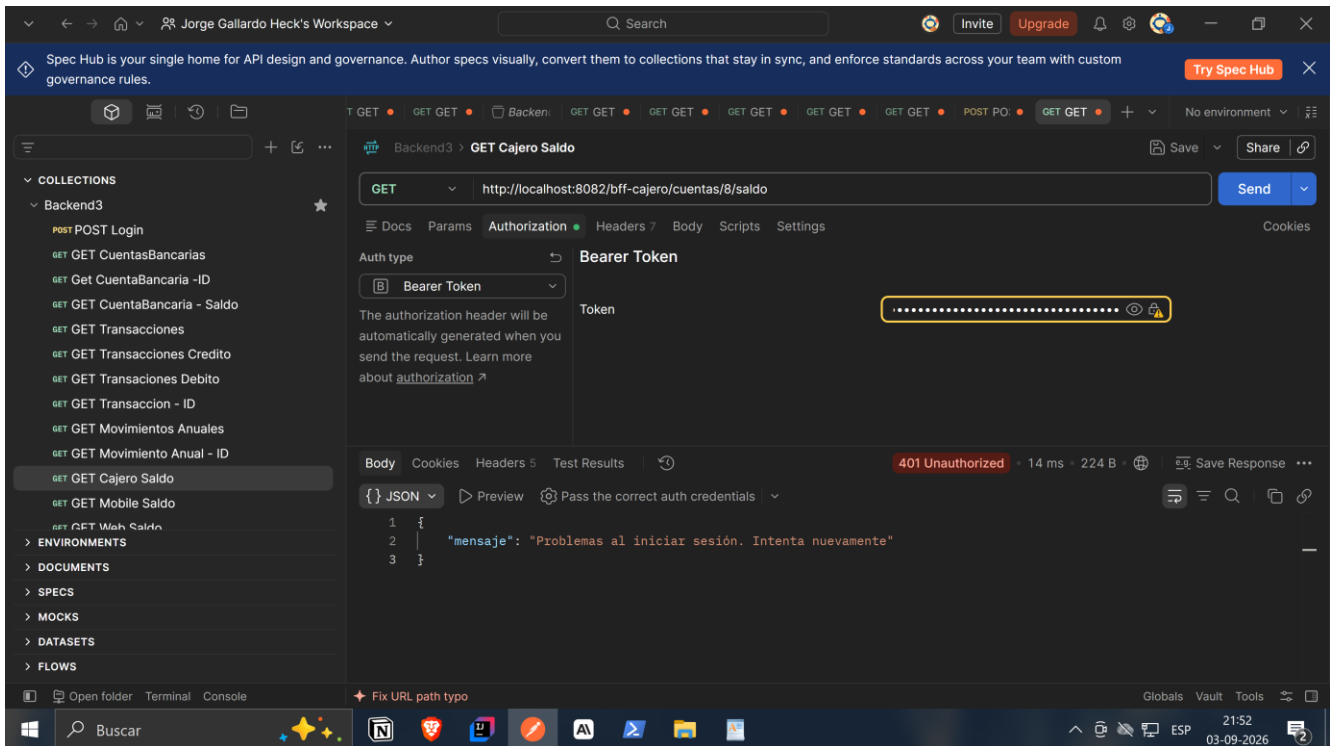
ID	id_usuario	cuenta_id_legacy	nombre	saldo	edad	tipo	interes	saldo_final
1	133	Alice Brown	12000.00	100	hipoteca	96.00	12006.00	
2	106	Jane Smith	12000.00	40	prestamo	180.00	12180.00	
3	122	Bob Johnson	7000.00	40	hipoteca	56.00	7056.00	
4	108	Jane Smith	12000.00	100	prestamo	180.00	12180.00	
5	124	Bob Johnson	12000.00	35	hipoteca	96.00	12006.00	
6	133	Steve Rogers	12000.00	40	prestamo	180.00	12180.00	
7	130	Diana Prince	12000.00	45	hipoteca	96.00	12006.00	
8	137	Steve Rogers	12000.00	30	ahorro	60.00	11260.00	
9	132	Jane Smith	10000.00	100	hipoteca	80.00	10080.00	



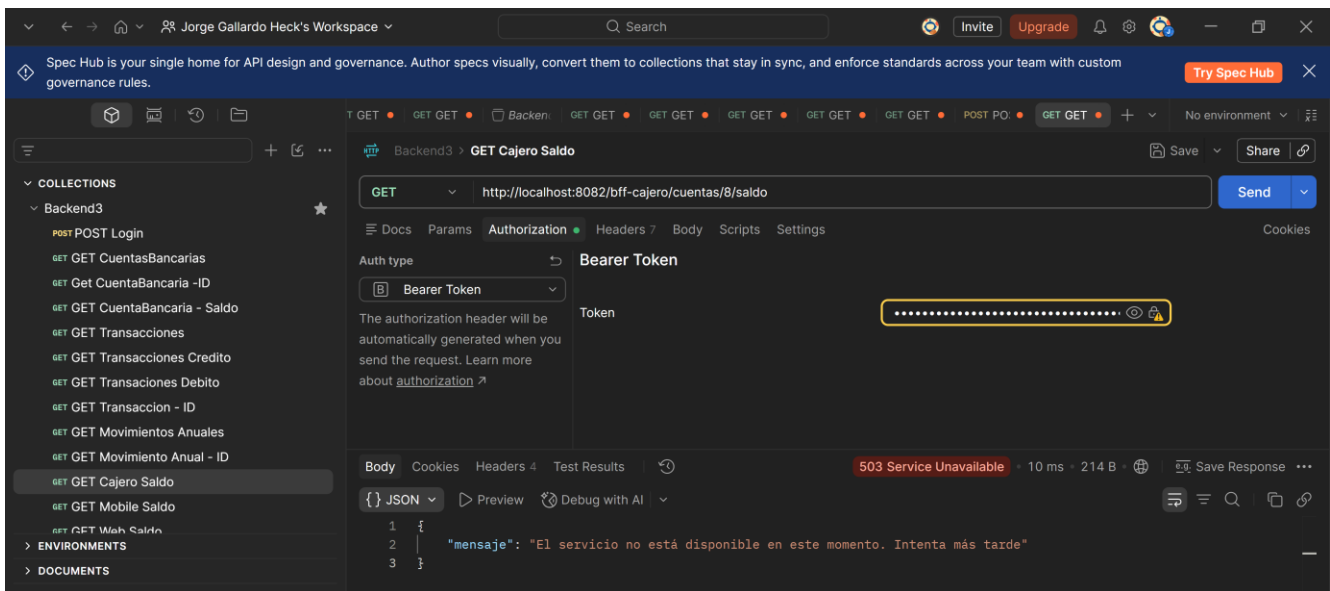
Sin id en el parámetro <http://localhost:8082/bff-cajero/cuentas//saldo>



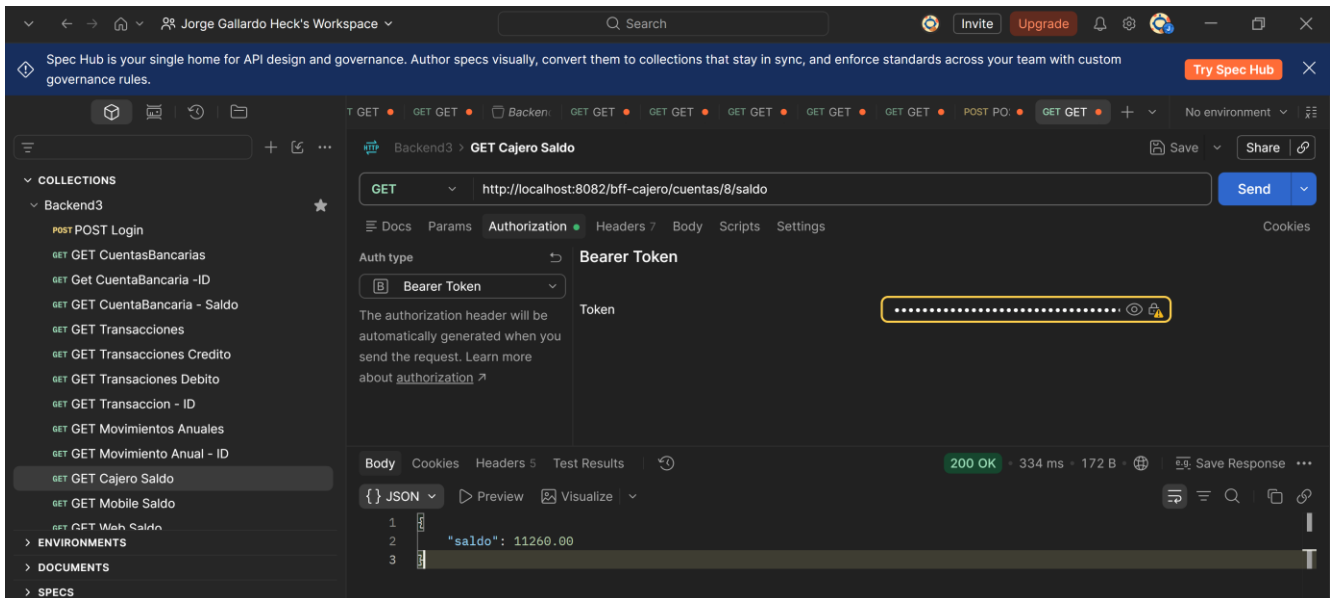
Token caducado <http://localhost:8082/bff-cajero/cuentas//saldo>



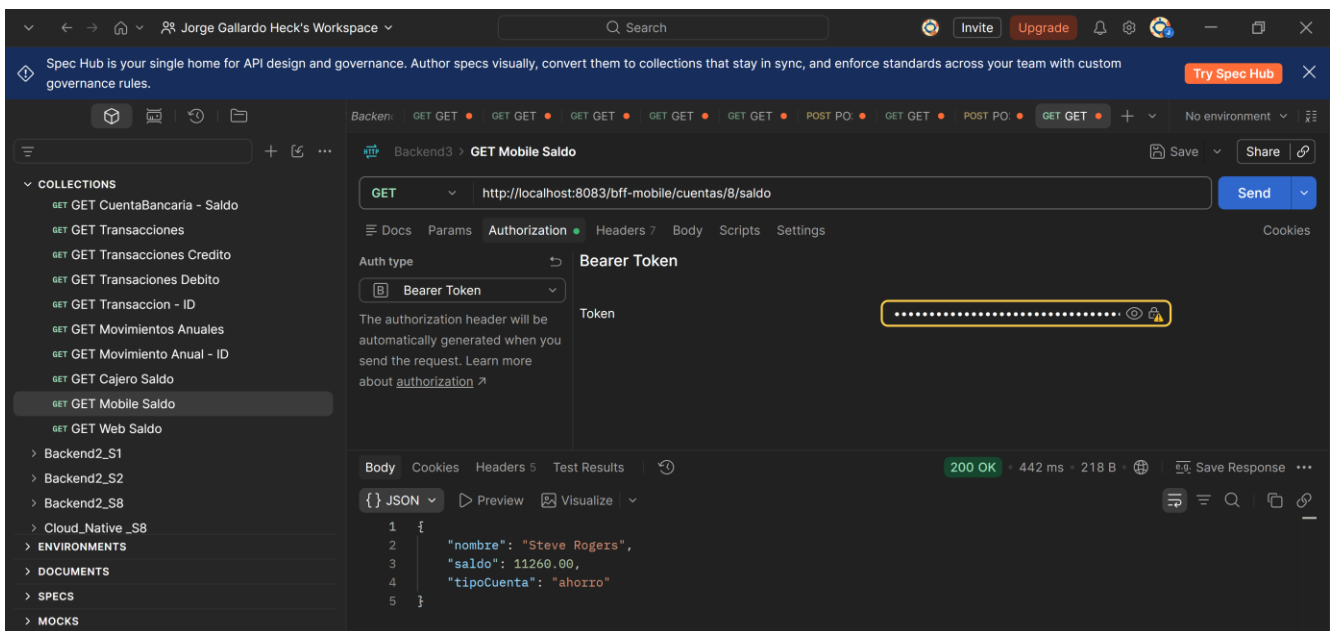
BancoCentral está caído



Respuesta Cajero todo correcto:



Respuesta Mobile:

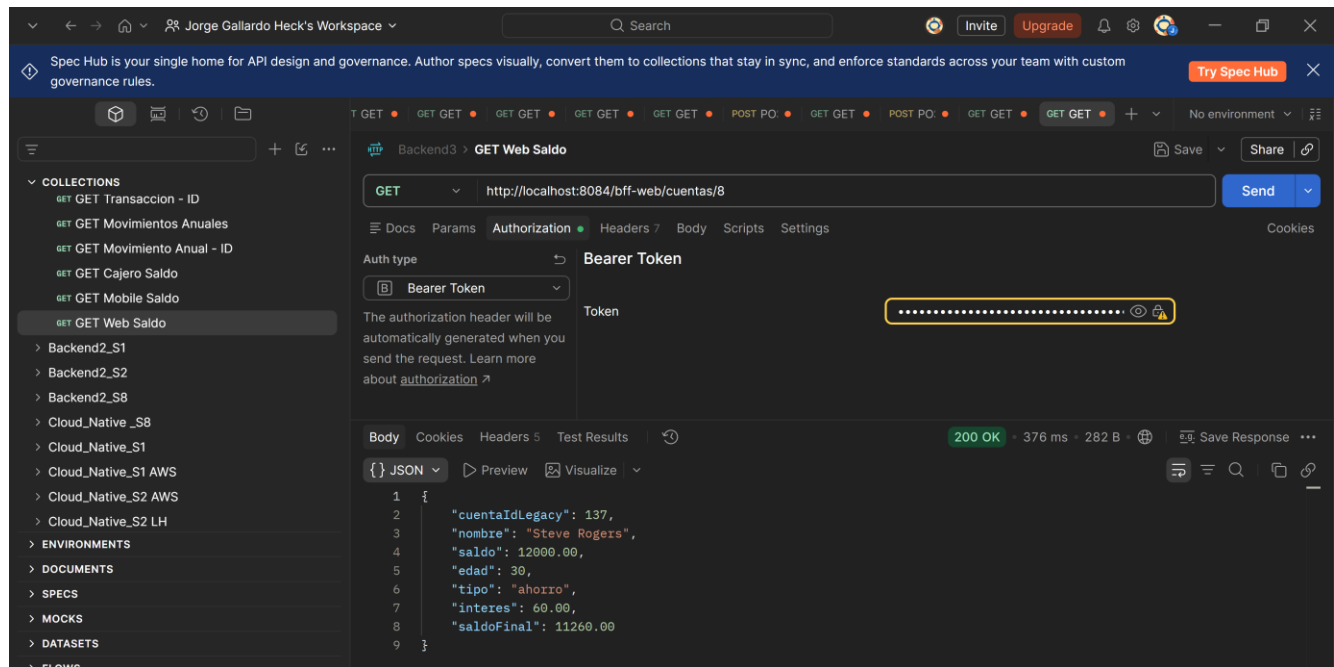


En una primera versión, BFF Mobile obtenía los datos de la cuenta consultando el endpoint de detalle completo del Backend Central (GET /api/cuentas/{id}, 7 campos), y descartaba 4 de ellos al construir su respuesta (nombre, saldo, tipo). Esto generaba tráfico de red innecesario entre el BFF y el Backend Central, contrario al objetivo de optimización de recursos que persigue el canal móvil.

Se corrigió agregando un endpoint específico en el Backend Central, GET /api/cuentas/{id}/mobile, que devuelve únicamente los 3 campos que el canal móvil necesita

(CuentaBancariaMobileDto). BFF Mobile fue ajustado para consumir este nuevo endpoint en lugar del de detalle completo, eliminando la transferencia de datos que no se utilizaban.

Respuesta Web:



Respuesta Web-Dashboard:

GET /bff-web/cuentas/{id}/dashboard

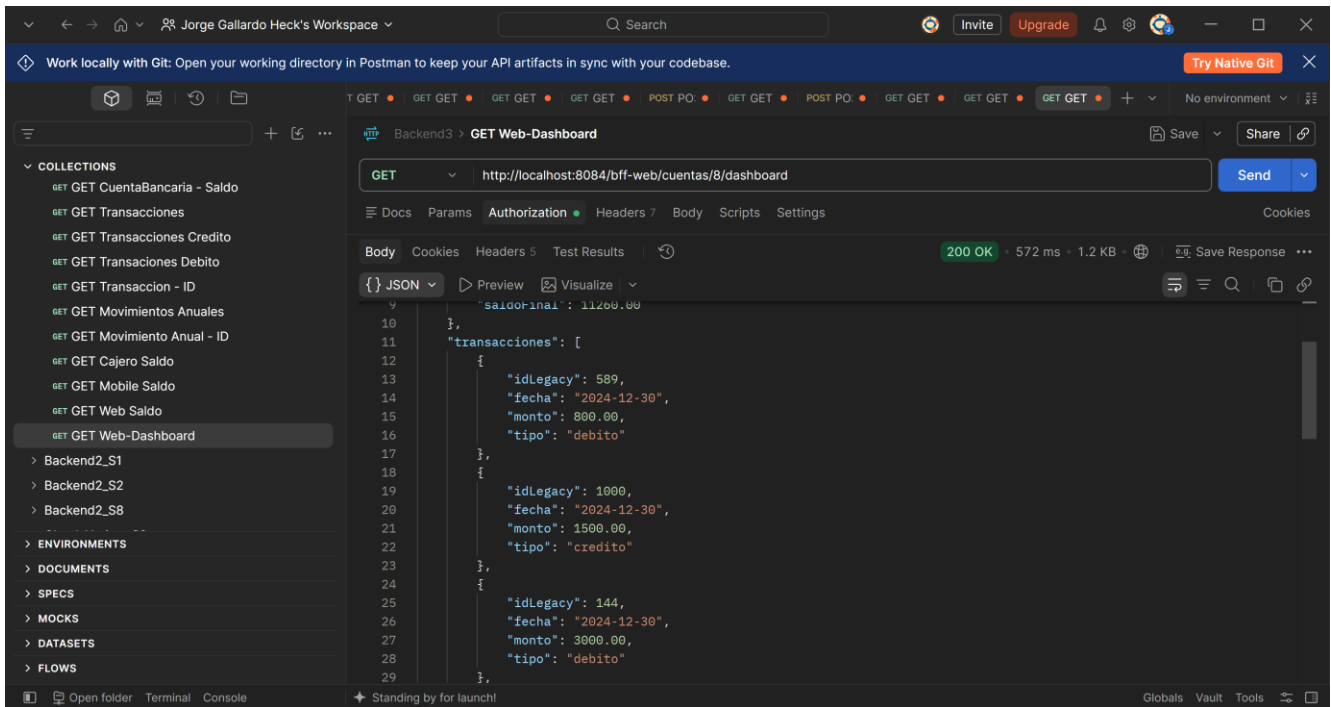
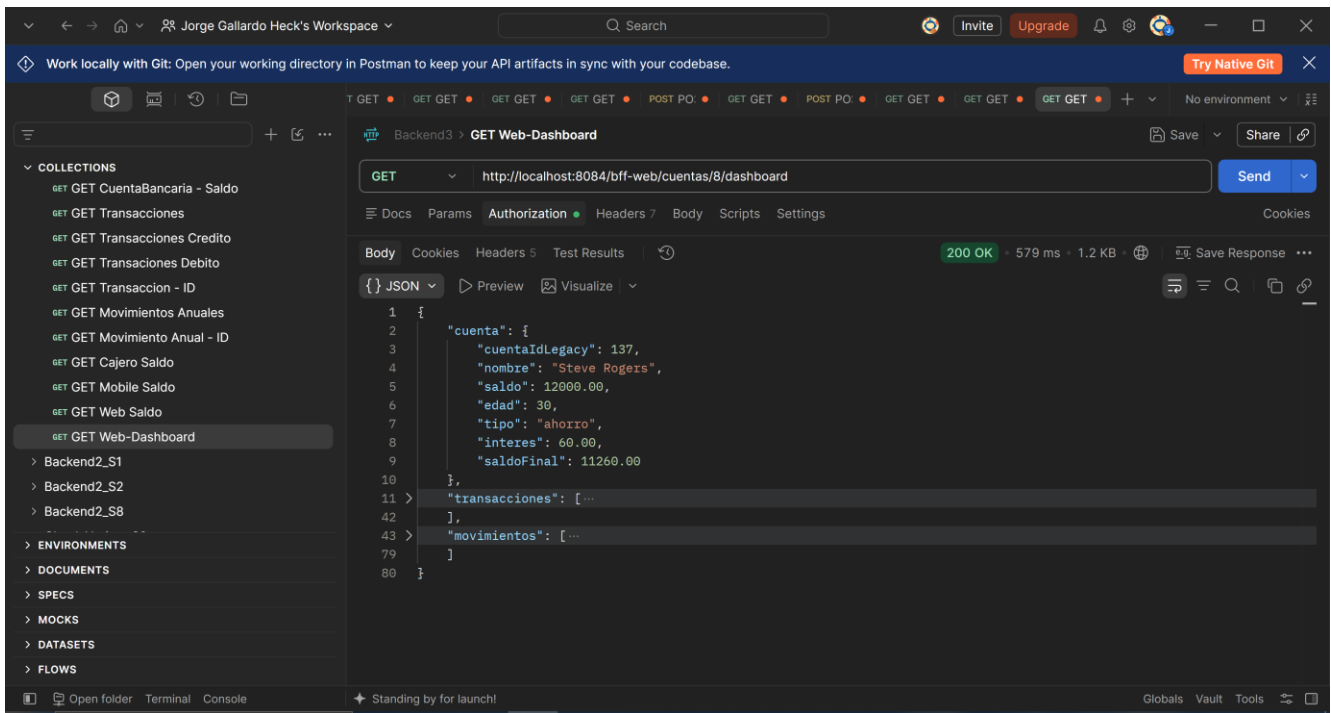
Descripción: entrega una vista consolidada de la cuenta, pensada para el resumen principal del canal web. Combina en una sola respuesta el detalle completo de la cuenta (nombre,

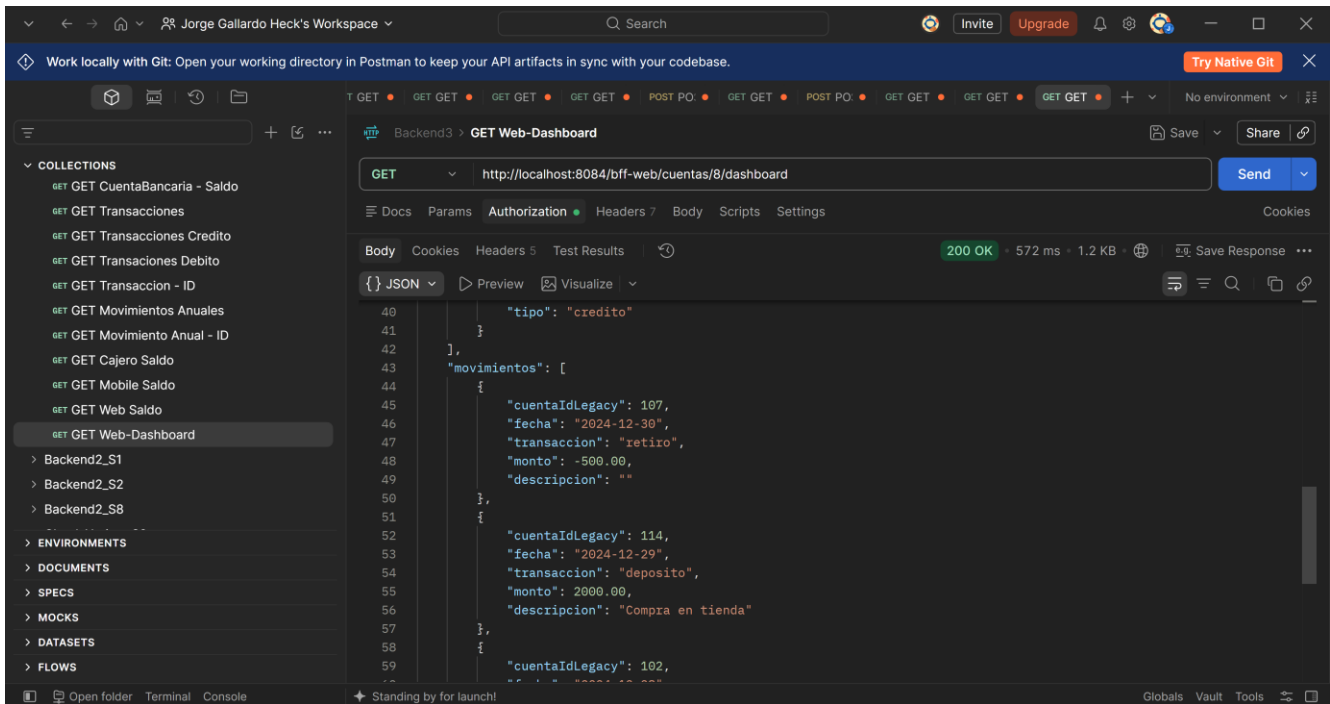
saldo, tipo, interés, saldo final) junto con sus 5 transacciones y 5 movimientos anuales más recientes, ordenados por fecha descendente, evitando que el cliente web deba hacer 3 llamadas separadas para armar esta vista.

Acceso: disponible tanto para CLIENTE como para EMPLEADO. La cuenta consultada valida propiedad (un CLIENTE solo puede ver el dashboard de su propia cuenta).

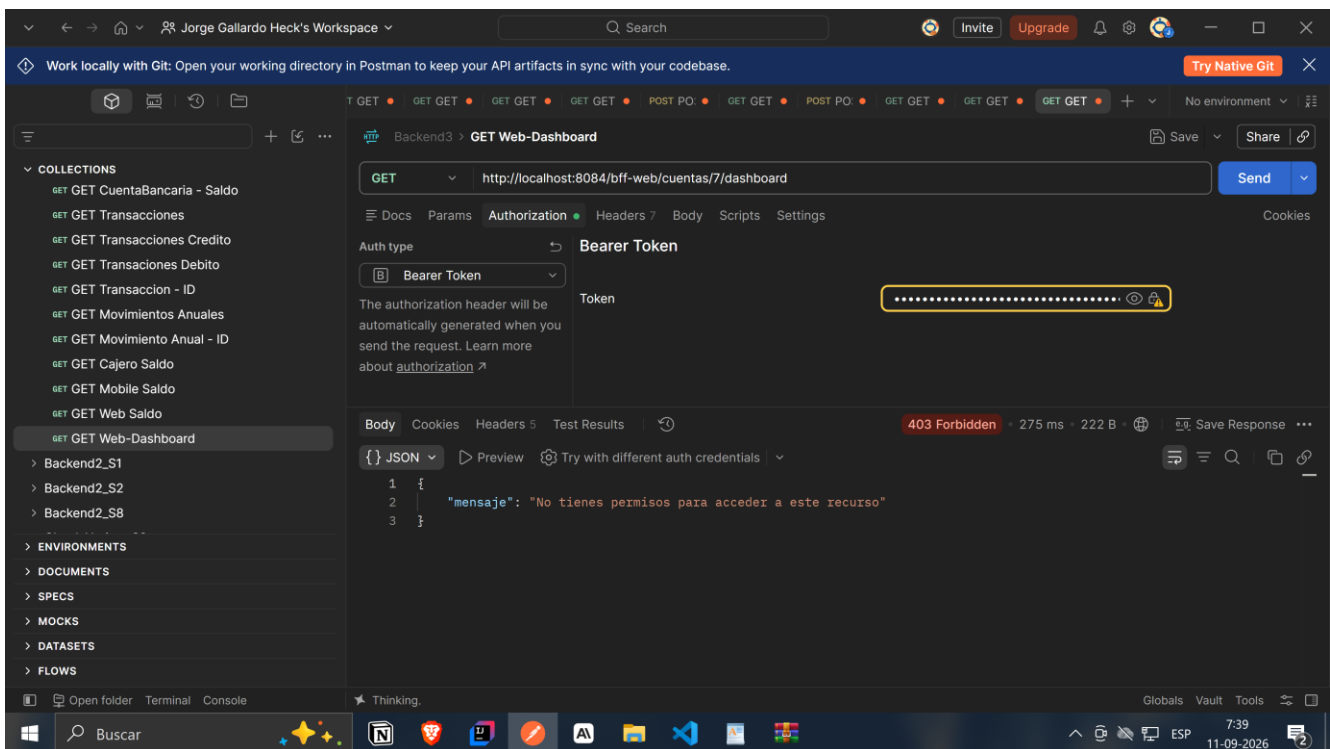
Caso de prueba: Steve, con rol CLIENTE, es dueño de la cuenta id=8 y puede consultar su dashboard sin problema; al intentar acceder al dashboard de la cuenta id=7 (que no le pertenece), recibe 403. María, con rol EMPLEADO, puede consultar el dashboard de cualquier cuenta del sistema, incluida la id=7 y la id=8.

Diferencia con @PreAuthorize: el endpoint de dashboard no usa @PreAuthorize porque fue diseñado para que tanto CLIENTE como EMPLEADO puedan acceder a él. Con @PreAuthorize("hasRole('EMPLEADO')"), un CLIENTE no podría ver las tablas de transacciones ni movimientos dentro del dashboard, ya que esa restricción existe en los endpoints originales de listado (/transacciones, /movimientos) precisamente porque, dentro de la lógica desarrollada, no es posible atribuir esas tablas a un cliente/usuario/cuenta en particular — no hay forma de filtrar "las transacciones de Steve" o "los movimientos de Steve", así que se optó por exponerlas como datos generales accesibles a cualquier usuario autenticado, en lugar de restringirlas solo a EMPLEADO.

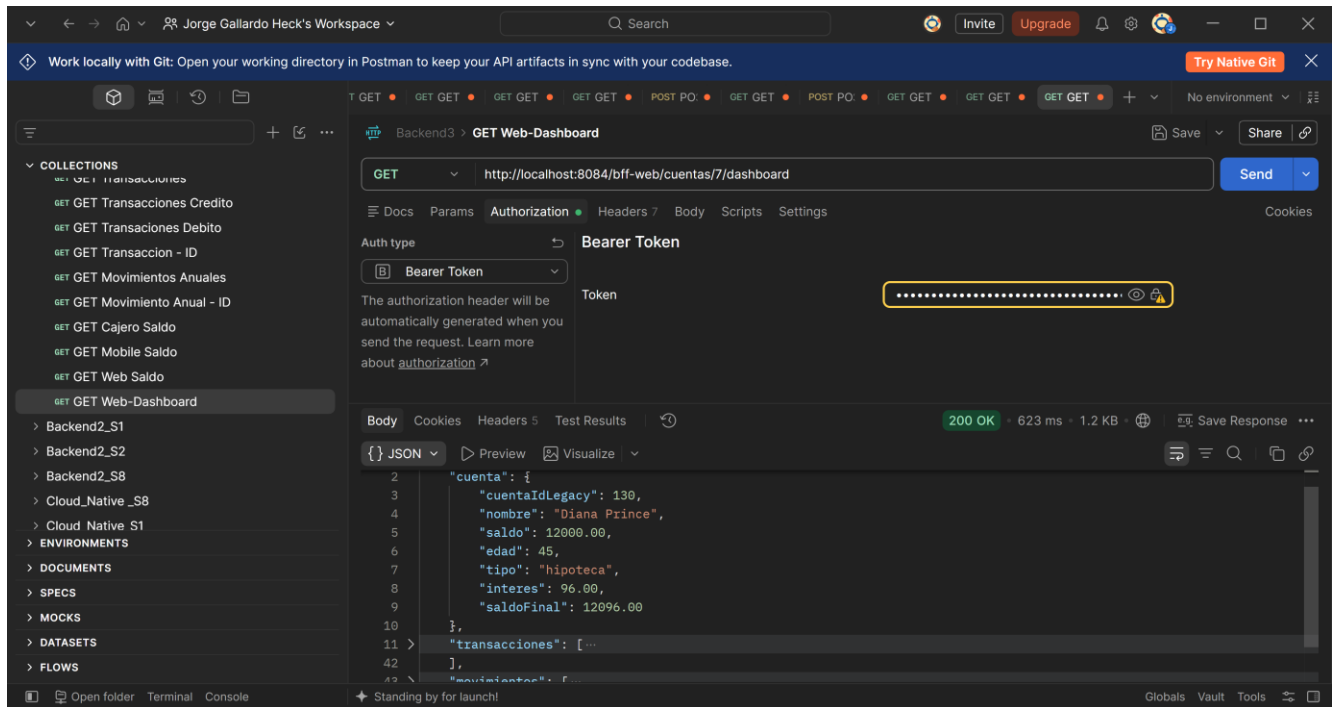




CLIENTE Steve cuenta ID=8 GET a cuenta ID=7



EMPLEADA Maria GET a cuenta ID=7



Implementación de hilos y logs para BFF-Web

Para completar la implementación del endpoint de dashboard, se incorporó el manejo de hilos con `CompletableFuture`, permitiendo que las tres fuentes de datos que lo componen (cuenta, transacciones y movimientos) se consulten en paralelo en lugar de secuencialmente. Esta decisión responde directamente a uno de los objetivos centrales del patrón BFF: optimizar el tiempo de respuesta hacia el cliente, evitando que la latencia total sea la suma de tres llamadas independientes cuando podría ser, en cambio, la de la más lenta de ellas.

Junto con esto, se agregó un registro de logs detallado en cada etapa del proceso — inicio y fin de cada consulta, identificando el hilo que la ejecuta, y el tiempo total transcurrido — no como una medida decorativa, sino como evidencia verificable de que el paralelismo efectivamente ocurre. Sin este registro, la ejecución concurrente sería una afirmación de diseño sin respaldo comprobable; con él, es posible observar directamente en consola que las tres tareas se inician casi al mismo tiempo, corren en hilos distintos, y finalizan de forma independiente.

```
2026-09-11T15:38:35.625-03:00 INFO 25860 --- [bff-web] [pool-2-thread-1] c.duoc.bff_web.service.DashboardService : [pool-2-thread-1] Consultando cuenta...
2026-09-11T15:38:35.626-03:00 INFO 25860 --- [bff-web] [pool-2-thread-2] c.duoc.bff_web.service.DashboardService : [pool-2-thread-2] Consultando transacciones...
2026-09-11T15:38:35.627-03:00 INFO 25860 --- [bff-web] [pool-2-thread-3] c.duoc.bff_web.service.DashboardService : [pool-2-thread-3] Consultando movimientos...
```

```
2026-09-11T15:38:35.625-03:00 INFO 25860 --- [bff-web] [nio-8084-exec-1] c.duoc.bff_web.service.DashboardService : Iniciando armado de dashboard para cuenta id=...
2026-09-11T15:38:35.625-03:00 INFO 25860 --- [bff-web] [pool-2-thread-1] c.duoc.bff_web.service.DashboardService : [pool-2-thread-1] Consultando cuenta...
2026-09-11T15:38:35.626-03:00 INFO 25860 --- [bff-web] [pool-2-thread-2] c.duoc.bff_web.service.DashboardService : [pool-2-thread-2] Consultando transacciones...
2026-09-11T15:38:35.627-03:00 INFO 25860 --- [bff-web] [pool-2-thread-3] c.duoc.bff_web.service.DashboardService : [pool-2-thread-3] Consultando movimientos...
2026-09-11T15:38:35.997-03:00 INFO 25860 --- [bff-web] [pool-2-thread-1] c.duoc.bff_web.service.DashboardService : [pool-2-thread-1] Cuenta obtenida
2026-09-11T15:38:36.019-03:00 INFO 25860 --- [bff-web] [pool-2-thread-2] c.duoc.bff_web.service.DashboardService : [pool-2-thread-2] Transacciones obtenidas
2026-09-11T15:38:36.040-03:00 INFO 25860 --- [bff-web] [pool-2-thread-3] c.duoc.bff_web.service.DashboardService : [pool-2-thread-3] Movimientos obtenidos
2026-09-11T15:38:36.040-03:00 INFO 25860 --- [bff-web] [nio-8084-exec-1] c.duoc.bff_web.service.DashboardService : Las 3 llamadas finalizaron en 417 ms, armando respuesta combinada
```

Se observa en los logs que las tres llamadas se inician a las 35.625, 35.626 y 35.627 respectivamente, con una diferencia de apenas 1 a 2 ms entre la primera y la última evidencia de que las tres tareas se lanzan de forma prácticamente simultánea, sin esperar a que la anterior finalice.

```
try {
    CompletableFuture.allOf(cuentaFuture, transaccionesFuture, movimientosFuture).join();
} catch (CompletionException ex) {
    if (ex.getCause() instanceof RuntimeException runtimeCause) {
        throw runtimeCause;
    }
    throw ex;
}
```

Las llamadas asíncronas están envueltas en un try-catch que captura CompletionException. El bloque intenta recuperar la causa original (por ejemplo, AccesoDenegadoException) y relanzarla tal cual, para que el GlobalExceptionHandler la reconozca igual que en el flujo secuencial.

Como respaldo, se agregó un manejador genérico de CompletionException en el GlobalExceptionHandler, que actúa solo si la causa real no pudo identificarse, evitando que un error inesperado llegue sin control al cliente.

```
@ExceptionHandler(CompletionException.class) no usages
public ResponseEntity<ErrorResponseDto> manejarErrorInterno(CompletionException e) {
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(new ErrorResponseDto( mensaje: "Ocurrió un error inesperado al procesar la solicitud"));
}
```



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.